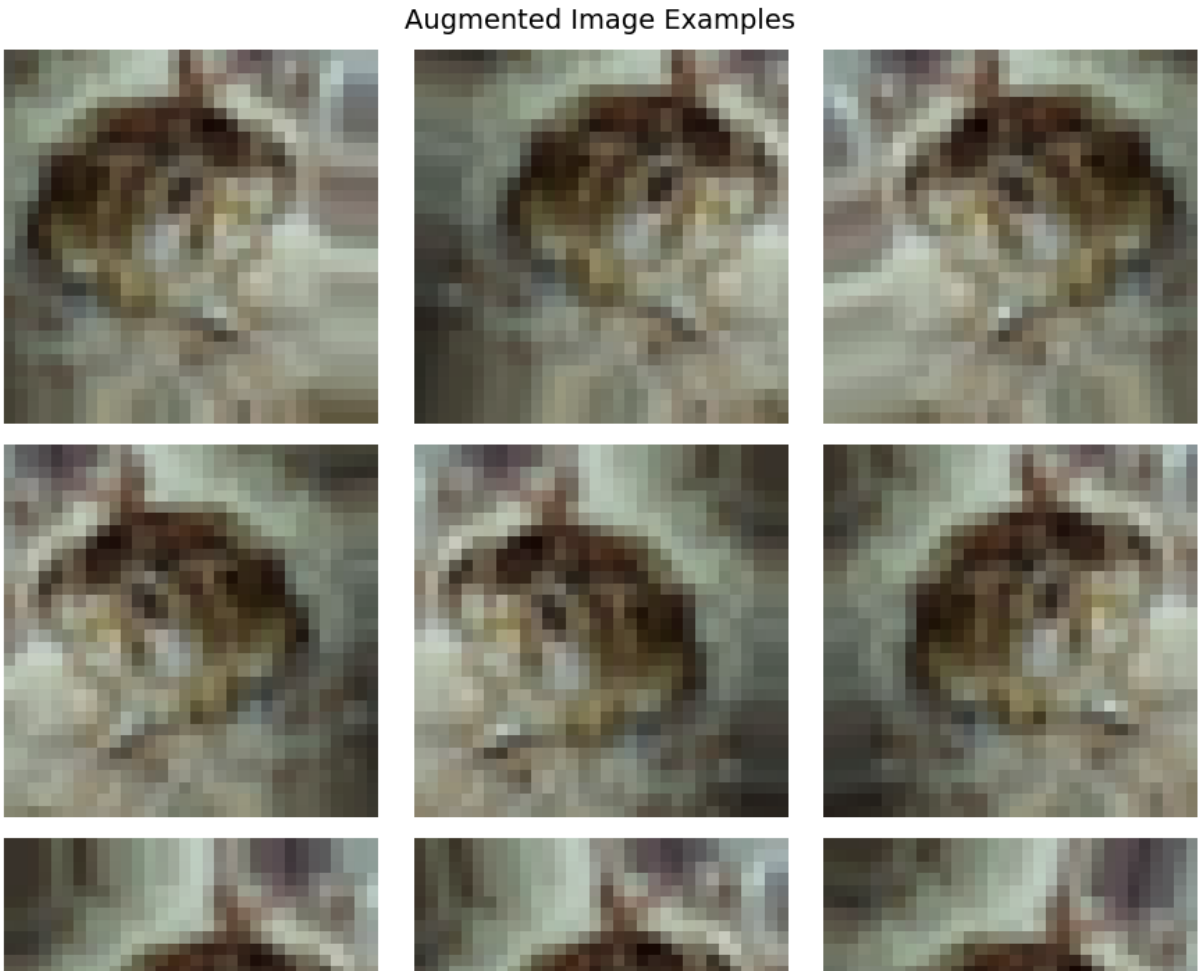
```
# ============================================================
# Section 1: Imports, Dataset Loading, Exploration, Split,
# and Preprocessing (CLEAN VERSION)
# ============================================================

# 1. Imports
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow version:", tf.__version__)

# ------------------------------------------------------------
# 2. Load CIFAR-10 dataset
# ------------------------------------------------------------
(x_train_full, y_train_full), (x_test, y_test) = cifar10.load_data()

print("\nOriginal dataset shapes:")
print("x_train_full:", x_train_full.shape)
print("y_train_full:", y_train_full.shape)
print("x_test:", x_test.shape)
print("y_test:", y_test.shape)

# ------------------------------------------------------------
# 3. Class names
# ------------------------------------------------------------
class_names = [
    'airplane', 'automobile', 'bird', 'cat', 'deer',
    'dog', 'frog', 'horse', 'ship', 'truck'
]

print("\nClass names:", class_names)
print("Number of classes:", len(class_names))

# ------------------------------------------------------------
# 4. Display sample images (FIXED VERSION)
# ------------------------------------------------------------
```

```
plt.figure(figsize=(12, 6))
for i in range(12):
    plt.subplot(3, 4, i + 1)
    plt.imshow(x_train_full[i])
    plt.title(class_names[y_train_full[i][0]])  # FIXED
    plt.axis('off')

plt.suptitle("Sample CIFAR-10 Images", fontsize=14)
plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# 5. Create Training / Validation Split
# ------------------------------------------------------------
x_train, x_val, y_train, y_val = train_test_split(
    x_train_full,
    y_train_full,
    test_size=10000,
    random_state=42,
    stratify=y_train_full
)

print("\nSplit dataset shapes:")
print("x_train:", x_train.shape, "y_train:", y_train.shape)
print("x_val:", x_val.shape, "y_val:", y_val.shape)
print("x_test:", x_test.shape, "y_test:", y_test.shape)

# ------------------------------------------------------------
# 6. Normalize pixel values
# ------------------------------------------------------------
x_train = x_train.astype("float32") / 255.0
x_val = x_val.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("\nPixel value range after normalization:")
print("x_train min:", x_train.min(), "x_train max:", x_train.max())

# ------------------------------------------------------------
# 7. One-hot encode labels
# ------------------------------------------------------------
y_train_cat = to_categorical(y_train, 10)
y_val_cat = to_categorical(y_val, 10)
y_test_cat = to_categorical(y_test, 10)

print("\nOne-hot encoded label shapes:")
print("y_train_cat:", y_train_cat.shape)
print("y_val_cat:", y_val_cat.shape)
print("y_test_cat:", y_test_cat.shape)
```

```
# ------------------------------------------------------------
# 8. Data Augmentation
# ------------------------------------------------------------
train_datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)

train_datagen.fit(x_train)

print("\nData augmentation configured successfully.")

# ------------------------------------------------------------
# 9. Visualize augmented images
# ------------------------------------------------------------
sample_image = x_train[0].reshape((1, 32, 32, 3))

plt.figure(figsize=(9, 9))
augmented_images = train_datagen.flow(sample_image, batch_size=1)

for i in range(9):
    plt.subplot(3, 3, i + 1)
    batch = next(augmented_images)
    plt.imshow(batch[0])
    plt.axis('off')

plt.suptitle("Augmented Image Examples", fontsize=14)
plt.tight_layout()
plt.show()
```

```
TensorFlow version: 2.19.0

Original dataset shapes:
x_train_full: (50000, 32, 32, 3)
y_train_full: (50000, 1)
x_test: (10000, 32, 32, 3)
y_test: (10000, 1)

Class names: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog'
Number of classes: 10
```

```
Split dataset shapes:
x_train: (40000, 32, 32, 3) y_train: (40000, 1)
x_val: (10000, 32, 32, 3) y_val: (10000, 1)
x_test: (10000, 32, 32, 3) y_test: (10000, 1)

Pixel value range after normalization:
x_train min: 0.0 x_train max: 1.0

One-hot encoded label shapes:
y_train_cat: (40000, 10)
y_val_cat: (10000, 10)
y_test_cat: (10000, 10)

Data augmentation configured successfully.
```



```
# ============================================================
# Section 2: Build and Train the Custom CNN Model
# ============================================================

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPla

# 1. Build the custom CNN model
```

```
cnn_model = Sequential([
    Input(shape=(32, 32, 3)),

    Conv2D(32, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    Conv2D(32, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),

    Conv2D(64, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.30),

    Conv2D(128, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.35),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.50),
    Dense(10, activation='softmax')
])

# 2. Compile the model
cnn_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 3. Show model summary
print("Custom CNN Model Summary:")
cnn_model.summary()

# 4. Define callbacks
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True,
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
```

```
    factor=0.5,
    patience=3,
    min_lr=1e-6,
    verbose=1
)

# 5. Train the model
cnn_history = cnn_model.fit(
    train_datagen.flow(x_train, y_train_cat, batch_size=64),
    epochs=25,
    validation_data=(x_val, y_val_cat),
    callbacks=[early_stopping, reduce_lr],
    verbose=1
)
```

```
Custom CNN Model Summary:
```

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 32, 32, 32)	
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	
conv2d_1 (Conv2D)	(None, 32, 32, 32)	
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	
dropout (Dropout)	(None, 16, 16, 32)	
conv2d_2 (Conv2D)	(None, 16, 16, 64)	
batch_normalization_1 (BatchNormalization)	(None, 16, 16, 64)	
conv2d_3 (Conv2D)	(None, 16, 16, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	
dropout_1 (Dropout)	(None, 8, 8, 64)	
conv2d_4 (Conv2D)	(None, 8, 8, 128)	
batch_normalization_2 (BatchNormalization)	(None, 8, 8, 128)	
conv2d_5 (Conv2D)	(None, 8, 8, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	
dropout_2 (Dropout)	(None, 4, 4, 128)	
flatten (Flatten)	(None, 2048)	

dense (Dense)	(None, 128)	
dropout_3 (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 10)	

Total params: 551,466 (2.10 MB)
Trainable params: 551,018 (2.10 MB)
Non-trainable params: 448 (1.75 KB)

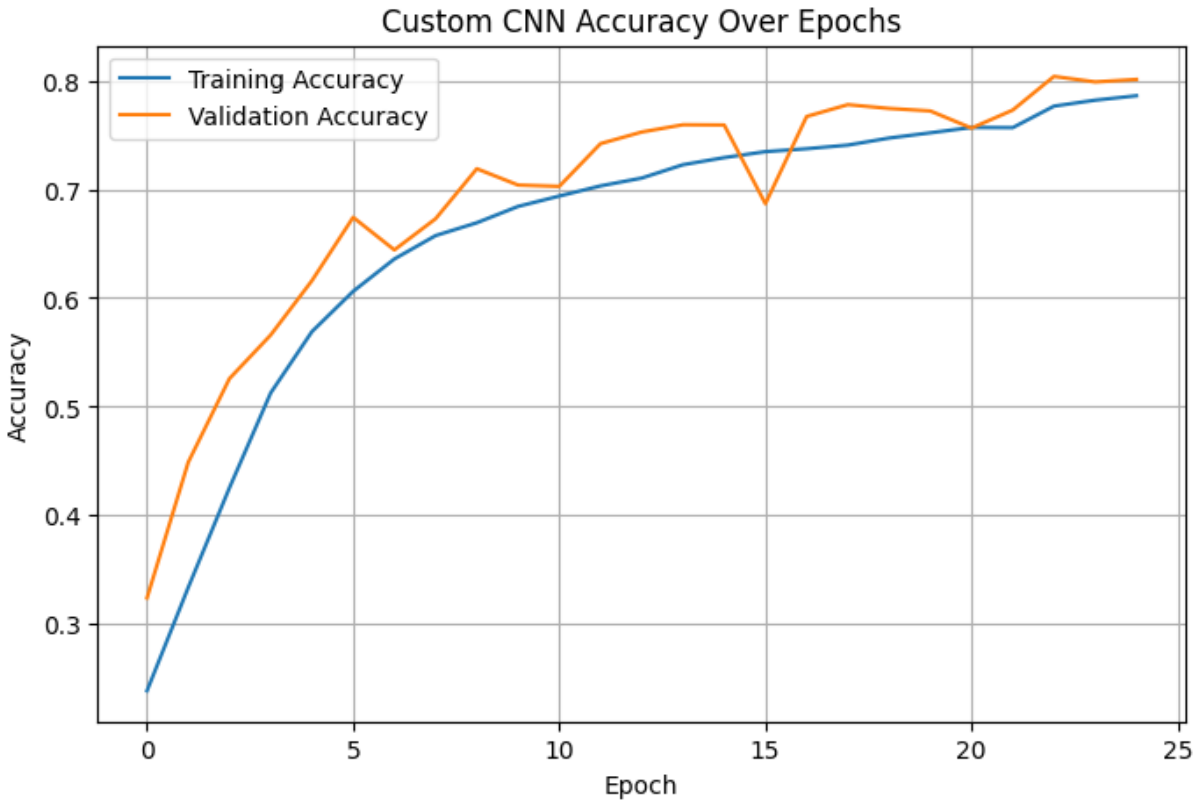
```
Epoch 1/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 320s 505ms/step - accuracy: 0.2386 - l
Epoch 2/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 319s 511ms/step - accuracy: 0.3337 - l
Epoch 3/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 318s 509ms/step - accuracy: 0.4254 - l
Epoch 4/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 318s 508ms/step - accuracy: 0.5127 - l
Epoch 5/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 314s 502ms/step - accuracy: 0.5691 - l
Epoch 6/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 316s 505ms/step - accuracy: 0.6062 - l
Epoch 7/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 318s 509ms/step - accuracy: 0.6360 - l
Epoch 8/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 312s 500ms/step - accuracy: 0.6575 - l
Epoch 9/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 319s 511ms/step - accuracy: 0.6693 - l
Epoch 10/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 315s 504ms/step - accuracy: 0.6843 - l
Epoch 11/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 311s 498ms/step - accuracy: 0.6939 - l
Epoch 12/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 311s 497ms/step - accuracy: 0.7032 - l
Epoch 13/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 310s 496ms/step - accuracy: 0.7104 - l
Epoch 14/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 325s 501ms/step - accuracy: 0.7227 - l
Epoch 15/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 309s 495ms/step - accuracy: 0.7293 - l
Epoch 16/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 311s 498ms/step - accuracy: 0.7347 - l
Epoch 17/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 323s 499ms/step - accuracy: 0.7374 - l
Epoch 18/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 311s 497ms/step - accuracy: 0.7408 - l
Epoch 19/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 316s 506ms/step - accuracy: 0.7473 - l
Epoch 20/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 311s 497ms/step - accuracy: 0.7521 - l
Epoch 21/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 316s 505ms/step - accuracy: 0.7570 - l
Epoch 22/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 0s 472ms/step - accuracy: 0.7549 - los
```

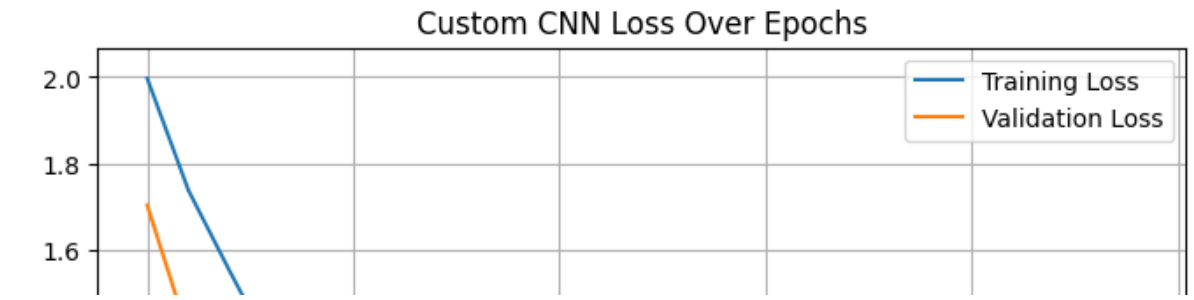
```
Epoch 22: ReduceLROnPlateau reducing learning rate to 0.00050000002
625/625 ━━━━━━━━━━━━━━━━━━━━ 311s 498ms/step - accuracy: 0.7569 - l
Epoch 23/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 315s 504ms/step - accuracy: 0.7766 - l
Epoch 24/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 311s 498ms/step - accuracy: 0.7821 - l
Epoch 25/25
625/625 ━━━━━━━━━━━━━━━━━━━━ 311s 497ms/step - accuracy: 0.7862 - l
Restoring model weights from the end of the best epoch: 23.
```

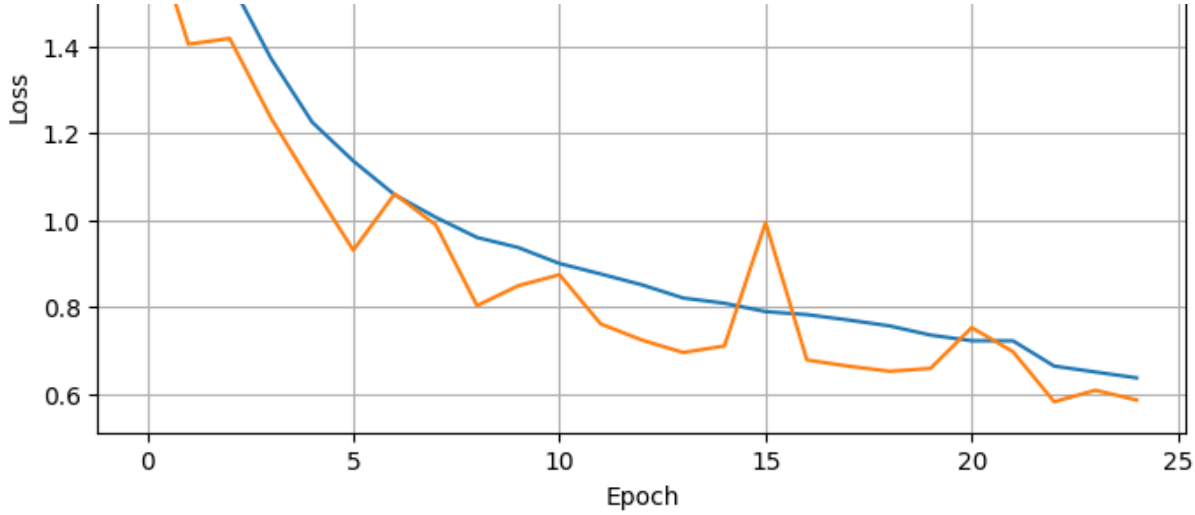
```
# ============================================================
# Section 3: Evaluate the Custom CNN Model
# ============================================================

from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# 1. Evaluate on the test set
cnn_test_loss, cnn_test_accuracy = cnn_model.evaluate(x_test, y_tes

print("\nCustom CNN Test Loss:", round(cnn_test_loss, 4))
print("Custom CNN Test Accuracy:", round(cnn_test_accuracy, 4))

# 2. Plot training and validation accuracy
plt.figure(figsize=(8, 5))
plt.plot(cnn_history.history['accuracy'], label='Training Accuracy'
plt.plot(cnn_history.history['val_accuracy'], label='Validation Acc
plt.title('Custom CNN Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# 3. Plot training and validation loss
plt.figure(figsize=(8, 5))
plt.plot(cnn_history.history['loss'], label='Training Loss')
plt.plot(cnn_history.history['val_loss'], label='Validation Loss')
plt.title('Custom CNN Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()

# 4. Generate predictions
y_pred_probs = cnn_model.predict(x_test)
y_pred = np.argmax(y_pred_probs, axis=1)
```

```
y_true = y_test.flatten()

# 5. Classification report
print("\nClassification Report for Custom CNN:\n")
print(classification_report(y_true, y_pred, target_names=class_name

# 6. Confusion matrix
cnn_cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cnn_cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix - Custom CNN')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

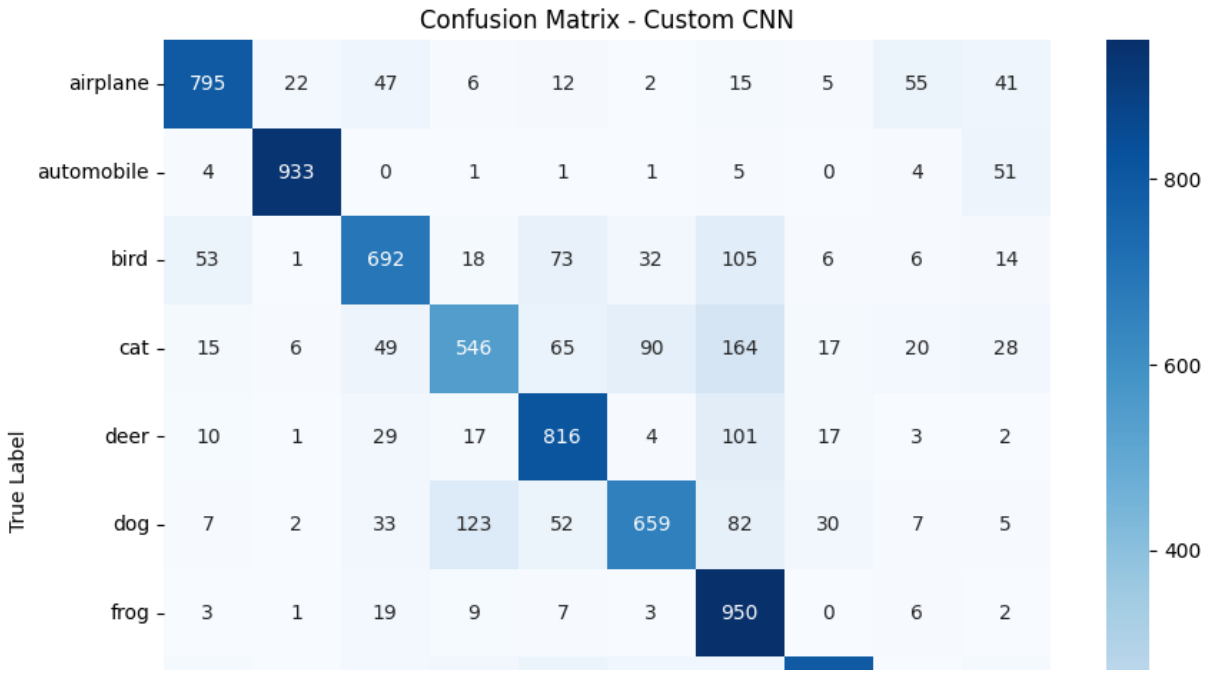
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 21s 65ms/step - accuracy: 0.8032 - loss

Custom CNN Test Loss: 0.6121
Custom CNN Test Accuracy: 0.8032
```

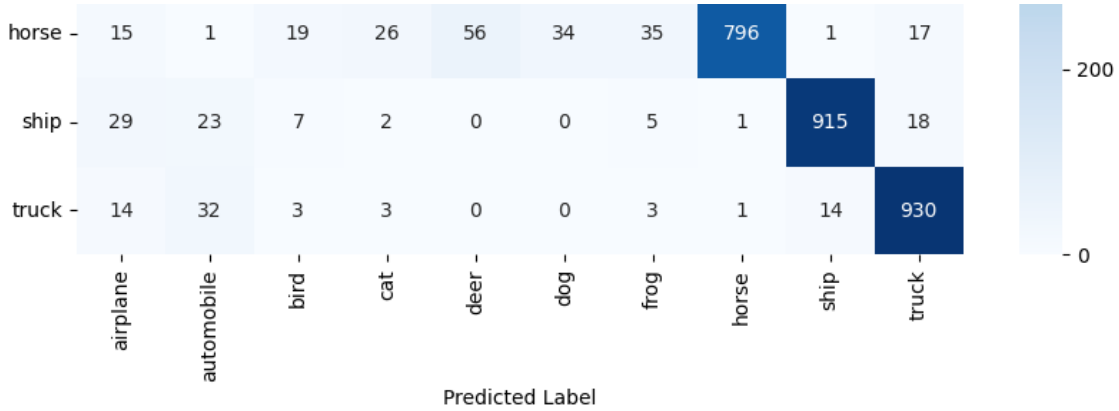







```
313/313 ━━━━━━━━━━━━━━━━━━━━ 18s 58ms/step

Classification Report for Custom CNN:

              precision    recall  f1-score   support

    airplane       0.84      0.80      0.82      1000
  automobile       0.91      0.93      0.92      1000
        bird       0.77      0.69      0.73      1000
         cat       0.73      0.55      0.62      1000
        deer       0.75      0.82      0.78      1000
         dog       0.80      0.66      0.72      1000
        frog       0.65      0.95      0.77      1000
       horse       0.91      0.80      0.85      1000
        ship       0.89      0.92      0.90      1000
       truck       0.84      0.93      0.88      1000

    accuracy                           0.80     10000
   macro avg       0.81      0.80      0.80     10000
weighted avg       0.81      0.80      0.80     10000
```






```
# ============================================================
# Section 4: Transfer Learning using MobileNetV2
# ============================================================

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import GlobalAveragePooling2D
from tensorflow.keras.models import Model

# 1. Resize images to fit MobileNetV2 input (96x96)
IMG_SIZE = 96

def resize_images(images):
    return tf.image.resize(images, (IMG_SIZE, IMG_SIZE))
```

```
x_train_resized = resize_images(x_train)
x_val_resized = resize_images(x_val)
x_test_resized = resize_images(x_test)

# 2. Load pretrained MobileNetV2 (without top layer)
base_model = MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights='imagenet'
)

# Freeze base model
base_model.trainable = False

# 3. Add custom classification head
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(10, activation='softmax')(x)

transfer_model = Model(inputs=base_model.input, outputs=output)

# 4. Compile
transfer_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 5. Summary
print("Transfer Learning Model Summary:")
transfer_model.summary()

# 6. Train (frozen base)
transfer_history = transfer_model.fit(
    x_train_resized, y_train_cat,
    validation_data=(x_val_resized, y_val_cat),
    epochs=10,
    batch_size=64,
    verbose=1
)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/ker
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Transfer Learning Model Summary:
Model: "functional_1"
```

Layer (type)	Output Shape	Param #	Connected

input_layer_1 (InputLayer)	(None, 96, 96, 3)	0	–
Conv1 (Conv2D)	(None, 48, 48, 32)	864	input_laye
bn_Conv1 (BatchNormalizatio…	(None, 48, 48, 32)	128	Conv1[0][0
Conv1_relu (ReLU)	(None, 48, 48, 32)	0	bn_Conv1[0
expanded_conv_dept… (DepthwiseConv2D)	(None, 48, 48, 32)	288	Conv1_relu
expanded_conv_dept… (BatchNormalizatio…	(None, 48, 48, 32)	128	expanded_c
expanded_conv_dept… (ReLU)	(None, 48, 48, 32)	0	expanded_c
expanded_conv_proj… (Conv2D)	(None, 48, 48, 16)	512	expanded_c
expanded_conv_proj… (BatchNormalizatio…	(None, 48, 48, 16)	64	expanded_c
block_1_expand (Conv2D)	(None, 48, 48, 96)	1,536	expanded_c
block_1_expand_BN (BatchNormalizatio…	(None, 48, 48, 96)	384	block_1_ex
block_1_expand_relu (ReLU)	(None, 48, 48, 96)	0	block_1_ex
block_1_pad (ZeroPadding2D)	(None, 49, 49, 96)	0	block_1_ex
block_1_depthwise (DepthwiseConv2D)	(None, 24, 24, 96)	864	block_1_pa
block_1_depthwise_… (BatchNormalizatio…	(None, 24, 24, 96)	384	block_1_de
block_1_depthwise_… (ReLU)	(None, 24, 24, 96)	0	block_1_de
block_1_project (Conv2D)	(None, 24, 24, 24)	2,304	block_1_de
block_1_project_BN (BatchNormalizatio…	(None, 24, 24, 24)	96	block_1_pr
block_2_expand (Conv2D)	(None, 24, 24, 144)	3,456	block_1_pr

block_2_expand_BN (BatchNormalizatio…	(None, 24, 24, 144)	576	block_2_ex
block_2_expand_relu (ReLU)	(None, 24, 24, 144)	0	block_2_ex
block_2_depthwise (DepthwiseConv2D)	(None, 24, 24, 144)	1,296	block_2_ex
block_2_depthwise_… (BatchNormalizatio…	(None, 24, 24, 144)	576	block_2_de
block_2_depthwise_… (ReLU)	(None, 24, 24, 144)	0	block_2_de
block_2_project (Conv2D)	(None, 24, 24, 24)	3,456	block_2_de
block_2_project_BN (BatchNormalizatio…	(None, 24, 24, 24)	96	block_2_pr
block_2_add (Add)	(None, 24, 24, 24)	0	block_1_pr block_2_pr
block_3_expand (Conv2D)	(None, 24, 24, 144)	3,456	block_2_ad
block_3_expand_BN (BatchNormalizatio…	(None, 24, 24, 144)	576	block_3_ex
block_3_expand_relu (ReLU)	(None, 24, 24, 144)	0	block_3_ex
block_3_pad (ZeroPadding2D)	(None, 25, 25, 144)	0	block_3_ex
block_3_depthwise (DepthwiseConv2D)	(None, 12, 12, 144)	1,296	block_3_pa
block_3_depthwise_… (BatchNormalizatio…	(None, 12, 12, 144)	576	block_3_de
block_3_depthwise_… (ReLU)	(None, 12, 12, 144)	0	block_3_de
block_3_project (Conv2D)	(None, 12, 12, 32)	4,608	block_3_de
block_3_project_BN (BatchNormalizatio…	(None, 12, 12, 32)	128	block_3_pr
block_4_expand (Conv2D)	(None, 12, 12, 192)	6,144	block_3_pr
block_4_expand_BN (BatchNormalizatio…	(None, 12, 12, 192)	768	block_4_ex

block_4_expand_relu (ReLU)	(None, 12, 12, 192)	0	block_4_ex
block_4_depthwise (DepthwiseConv2D)	(None, 12, 12, 192)	1,728	block_4_ex
block_4_depthwise_… (BatchNormalizatio…	(None, 12, 12, 192)	768	block_4_de
block_4_depthwise_… (ReLU)	(None, 12, 12, 192)	0	block_4_de
block_4_project (Conv2D)	(None, 12, 12, 32)	6,144	block_4_de
block_4_project_BN (BatchNormalizatio…	(None, 12, 12, 32)	128	block_4_pr
block_4_add (Add)	(None, 12, 12, 32)	0	block_3_pr block_4_pr
block_5_expand (Conv2D)	(None, 12, 12, 192)	6,144	block_4_ad
block_5_expand_BN (BatchNormalizatio…	(None, 12, 12, 192)	768	block_5_ex
block_5_expand_relu (ReLU)	(None, 12, 12, 192)	0	block_5_ex
block_5_depthwise (DepthwiseConv2D)	(None, 12, 12, 192)	1,728	block_5_ex
block_5_depthwise_… (BatchNormalizatio…	(None, 12, 12, 192)	768	block_5_de
block_5_depthwise_… (ReLU)	(None, 12, 12, 192)	0	block_5_de
block_5_project (Conv2D)	(None, 12, 12, 32)	6,144	block_5_de
block_5_project_BN (BatchNormalizatio…	(None, 12, 12, 32)	128	block_5_pr
block_5_add (Add)	(None, 12, 12, 32)	0	block_4_ad block_5_pr
block_6_expand (Conv2D)	(None, 12, 12, 192)	6,144	block_5_ad
block_6_expand_BN (BatchNormalizatio…	(None, 12, 12, 192)	768	block_6_ex
block_6_expand_relu (ReLU)	(None, 12, 12, 192)	0	block_6_ex

block_6_pad (ZeroPadding2D)	(None, 13, 13, 192)	0	block_6_e
block_6_depthwise (DepthwiseConv2D)	(None, 6, 6, 192)	1,728	block_6_pa
block_6_depthwise_… (BatchNormalizatio…	(None, 6, 6, 192)	768	block_6_de
block_6_depthwise_… (ReLU)	(None, 6, 6, 192)	0	block_6_de
block_6_project (Conv2D)	(None, 6, 6, 64)	12,288	block_6_de
block_6_project_BN (BatchNormalizatio…	(None, 6, 6, 64)	256	block_6_pr
block_7_expand (Conv2D)	(None, 6, 6, 384)	24,576	block_6_pr
block_7_expand_BN (BatchNormalizatio…	(None, 6, 6, 384)	1,536	block_7_ex
block_7_expand_relu (ReLU)	(None, 6, 6, 384)	0	block_7_ex
block_7_depthwise (DepthwiseConv2D)	(None, 6, 6, 384)	3,456	block_7_ex
block_7_depthwise_… (BatchNormalizatio…	(None, 6, 6, 384)	1,536	block_7_de
block_7_depthwise_… (ReLU)	(None, 6, 6, 384)	0	block_7_de
block_7_project (Conv2D)	(None, 6, 6, 64)	24,576	block_7_de
block_7_project_BN (BatchNormalizatio…	(None, 6, 6, 64)	256	block_7_pr
block_7_add (Add)	(None, 6, 6, 64)	0	block_6_pr block_7_pr
block_8_expand (Conv2D)	(None, 6, 6, 384)	24,576	block_7_ad
block_8_expand_BN (BatchNormalizatio…	(None, 6, 6, 384)	1,536	block_8_ex
block_8_expand_relu (ReLU)	(None, 6, 6, 384)	0	block_8_ex
block_8_depthwise (DepthwiseConv2D)	(None, 6, 6, 384)	3,456	block_8_ex
block_8_depthwise	(None, 6, 6, 384)	1,536	block_8_de

block_8_depthwise_… (BatchNormalizatio…	(None, 6, 6, 384)	1,536	block_8_d
block_8_depthwise_… (ReLU)	(None, 6, 6, 384)	0	block_8_de
block_8_project (Conv2D)	(None, 6, 6, 64)	24,576	block_8_de
block_8_project_BN (BatchNormalizatio…	(None, 6, 6, 64)	256	block_8_pr
block_8_add (Add)	(None, 6, 6, 64)	0	block_7_ad block_8_pr
block_9_expand (Conv2D)	(None, 6, 6, 384)	24,576	block_8_ad
block_9_expand_BN (BatchNormalizatio…	(None, 6, 6, 384)	1,536	block_9_ex
block_9_expand_relu (ReLU)	(None, 6, 6, 384)	0	block_9_ex
block_9_depthwise (DepthwiseConv2D)	(None, 6, 6, 384)	3,456	block_9_ex
block_9_depthwise_… (BatchNormalizatio…	(None, 6, 6, 384)	1,536	block_9_de
block_9_depthwise_… (ReLU)	(None, 6, 6, 384)	0	block_9_de
block_9_project (Conv2D)	(None, 6, 6, 64)	24,576	block_9_de
block_9_project_BN (BatchNormalizatio…	(None, 6, 6, 64)	256	block_9_pr
block_9_add (Add)	(None, 6, 6, 64)	0	block_8_ad block_9_pr
block_10_expand (Conv2D)	(None, 6, 6, 384)	24,576	block_9_ad
block_10_expand_BN (BatchNormalizatio…	(None, 6, 6, 384)	1,536	block_10_e
block_10_expand_re… (ReLU)	(None, 6, 6, 384)	0	block_10_e
block_10_depthwise (DepthwiseConv2D)	(None, 6, 6, 384)	3,456	block_10_e
block_10_depthwise… (BatchNormalizatio…	(None, 6, 6, 384)	1,536	block_10_d
block_10_depthwise…	(None, 6, 6, 384)	0	block_10_d

(ReLU)			
block_10_project (Conv2D)	(None, 6, 6, 96)	36,864	block_10_
block_10_project_BN (BatchNormalizatio…	(None, 6, 6, 96)	384	block_10_
block_11_expand (Conv2D)	(None, 6, 6, 576)	55,296	block_10_
block_11_expand_BN (BatchNormalizatio…	(None, 6, 6, 576)	2,304	block_11_
block_11_expand_re… (ReLU)	(None, 6, 6, 576)	0	block_11_
block_11_depthwise (DepthwiseConv2D)	(None, 6, 6, 576)	5,184	block_11_
block_11_depthwise… (BatchNormalizatio…	(None, 6, 6, 576)	2,304	block_11_
block_11_depthwise… (ReLU)	(None, 6, 6, 576)	0	block_11_
block_11_project (Conv2D)	(None, 6, 6, 96)	55,296	block_11_
block_11_project_BN (BatchNormalizatio…	(None, 6, 6, 96)	384	block_11_
block_11_add (Add)	(None, 6, 6, 96)	0	block_10_ block_11_
block_12_expand (Conv2D)	(None, 6, 6, 576)	55,296	block_11_
block_12_expand_BN (BatchNormalizatio…	(None, 6, 6, 576)	2,304	block_12_
block_12_expand_re… (ReLU)	(None, 6, 6, 576)	0	block_12_
block_12_depthwise (DepthwiseConv2D)	(None, 6, 6, 576)	5,184	block_12_
block_12_depthwise… (BatchNormalizatio…	(None, 6, 6, 576)	2,304	block_12_
block_12_depthwise… (ReLU)	(None, 6, 6, 576)	0	block_12_
block_12_project (Conv2D)	(None, 6, 6, 96)	55,296	block_12_
block_12_project_BN (BatchNormalizatio…	(None, 6, 6, 96)	384	block_12_

(BatchNormalizatio…			
block_12_add (Add)	(None, 6, 6, 96)	0	block_11_a block_12_p
block_13_expand (Conv2D)	(None, 6, 6, 576)	55,296	block_12_a
block_13_expand_BN (BatchNormalizatio…	(None, 6, 6, 576)	2,304	block_13_e
block_13_expand_re… (ReLU)	(None, 6, 6, 576)	0	block_13_e
block_13_pad (ZeroPadding2D)	(None, 7, 7, 576)	0	block_13_e
block_13_depthwise (DepthwiseConv2D)	(None, 3, 3, 576)	5,184	block_13_p
block_13_depthwise… (BatchNormalizatio…	(None, 3, 3, 576)	2,304	block_13_d
block_13_depthwise… (ReLU)	(None, 3, 3, 576)	0	block_13_d
block_13_project (Conv2D)	(None, 3, 3, 160)	92,160	block_13_d
block_13_project_BN (BatchNormalizatio…	(None, 3, 3, 160)	640	block_13_p
block_14_expand (Conv2D)	(None, 3, 3, 960)	153,600	block_13_p
block_14_expand_BN (BatchNormalizatio…	(None, 3, 3, 960)	3,840	block_14_e
block_14_expand_re… (ReLU)	(None, 3, 3, 960)	0	block_14_e
block_14_depthwise (DepthwiseConv2D)	(None, 3, 3, 960)	8,640	block_14_e
block_14_depthwise… (BatchNormalizatio…	(None, 3, 3, 960)	3,840	block_14_d
block_14_depthwise… (ReLU)	(None, 3, 3, 960)	0	block_14_d
block_14_project (Conv2D)	(None, 3, 3, 160)	153,600	block_14_d
block_14_project_BN (BatchNormalizatio…	(None, 3, 3, 160)	640	block_14_p
block_14_add (Add)	(None, 3, 3, 160)	0	block_13_p block_14_p

block_15_expand (Conv2D)	(None, 3, 3, 960)	153,600	block_14_a
block_15_expand_BN (BatchNormalizatio…	(None, 3, 3, 960)	3,840	block_15_e
block_15_expand_re… (ReLU)	(None, 3, 3, 960)	0	block_15_e
block_15_depthwise (DepthwiseConv2D)	(None, 3, 3, 960)	8,640	block_15_e
block_15_depthwise… (BatchNormalizatio…	(None, 3, 3, 960)	3,840	block_15_d
block_15_depthwise… (ReLU)	(None, 3, 3, 960)	0	block_15_d
block_15_project (Conv2D)	(None, 3, 3, 160)	153,600	block_15_d
block_15_project_BN (BatchNormalizatio…	(None, 3, 3, 160)	640	block_15_p
block_15_add (Add)	(None, 3, 3, 160)	0	block_14_a block_15_p
block_16_expand (Conv2D)	(None, 3, 3, 960)	153,600	block_15_a
block_16_expand_BN (BatchNormalizatio…	(None, 3, 3, 960)	3,840	block_16_e
block_16_expand_re… (ReLU)	(None, 3, 3, 960)	0	block_16_e
block_16_depthwise (DepthwiseConv2D)	(None, 3, 3, 960)	8,640	block_16_e
block_16_depthwise… (BatchNormalizatio…	(None, 3, 3, 960)	3,840	block_16_d
block_16_depthwise… (ReLU)	(None, 3, 3, 960)	0	block_16_d
block_16_project (Conv2D)	(None, 3, 3, 320)	307,200	block_16_d
block_16_project_BN (BatchNormalizatio…	(None, 3, 3, 320)	1,280	block_16_p
Conv_1 (Conv2D)	(None, 3, 3, 1280)	409,600	block_16_p
Conv_1_bn (BatchNormalizatio…	(None, 3, 3, 1280)	5,120	Conv_1[0]

out_relu (ReLU)	(None, 3, 3, 1280)	0	Conv_1_bn
global_average_poo… (GlobalAveragePool…	(None, 1280)	0	out_relu[
dense_2 (Dense)	(None, 128)	163,968	global_ave
dropout_4 (Dropout)	(None, 128)	0	dense_2[0
dense_3 (Dense)	(None, 10)	1,290	dropout_4

Total params: 2,423,242 (9.24 MB)
Trainable params: 165,258 (645.54 KB)
Non-trainable params: 2,257,984 (8.61 MB)

```
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 301s 472ms/step - accuracy: 0.6656 - l
Epoch 2/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 319s 510ms/step - accuracy: 0.7450 - l
Epoch 3/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 321s 509ms/step - accuracy: 0.7611 - l
Epoch 4/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 319s 510ms/step - accuracy: 0.7743 - l
Epoch 5/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 298s 472ms/step - accuracy: 0.7839 - l
Epoch 6/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 294s 471ms/step - accuracy: 0.7900 - l
Epoch 7/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 323s 472ms/step - accuracy: 0.7954 - l
Epoch 8/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 320s 513ms/step - accuracy: 0.8022 - l
Epoch 9/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 316s 506ms/step - accuracy: 0.8034 - l
Epoch 10/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 295s 472ms/step - accuracy: 0.8123 - l
```

```
# ============================================================
# Section 5: Fine-Tune the Transfer Learning Model
# ============================================================

# Unfreeze the top layers of the base model
base_model.trainable = True

# Freeze most layers and fine-tune only the last 30 layers
for layer in base_model.layers[:-30]:
    layer.trainable = False

# Recompile with a lower learning rate for fine-tuning
transfer_model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("Number of trainable layers after fine-tuning:")
print(sum([layer.trainable for layer in transfer_model.layers]))

# Fine-tune the model
fine_tune_history = transfer_model.fit(
    x_train_resized, y_train_cat,
    validation_data=(x_val_resized, y_val_cat),
    epochs=5,
    batch_size=64,
    verbose=1
)
```

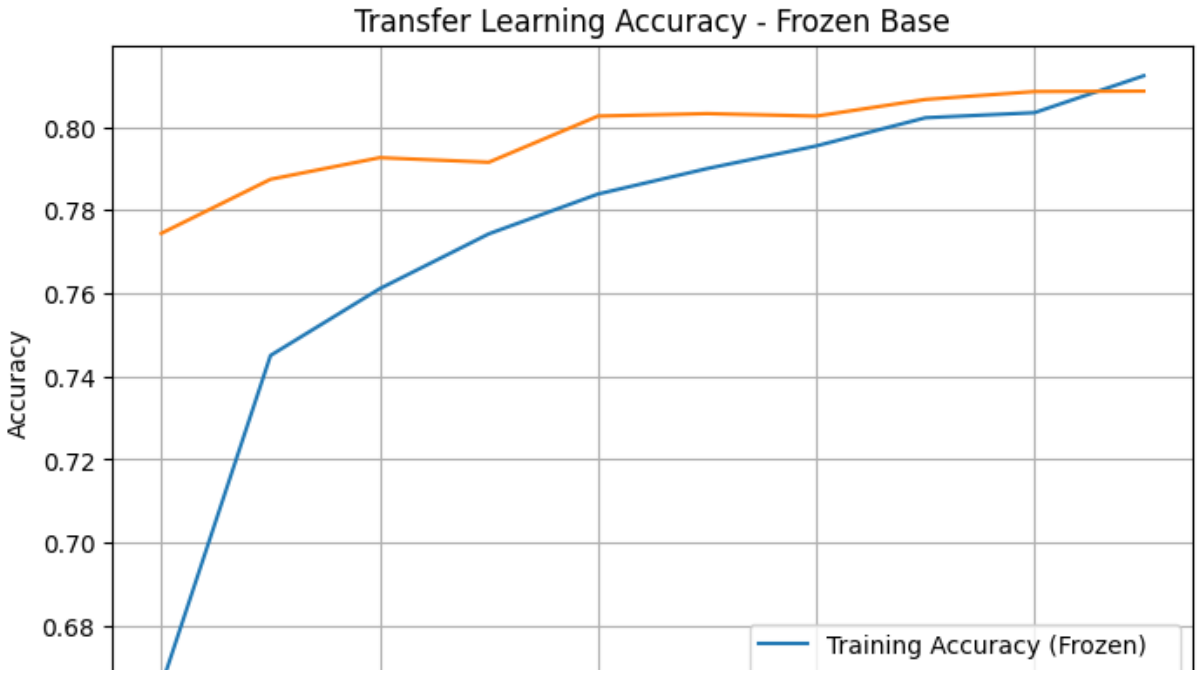
```
Number of trainable layers after fine-tuning:
34
Epoch 1/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 471s 737ms/step - accuracy: 0.6808 - lo
Epoch 2/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 458s 733ms/step - accuracy: 0.7561 - lo
Epoch 3/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 459s 735ms/step - accuracy: 0.7832 - lo
Epoch 4/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 460s 736ms/step - accuracy: 0.8112 - lo
Epoch 5/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 459s 734ms/step - accuracy: 0.8277 - lo
```

```
# ============================================================
# Section 6: Evaluate the Transfer Learning Model
# ============================================================
```

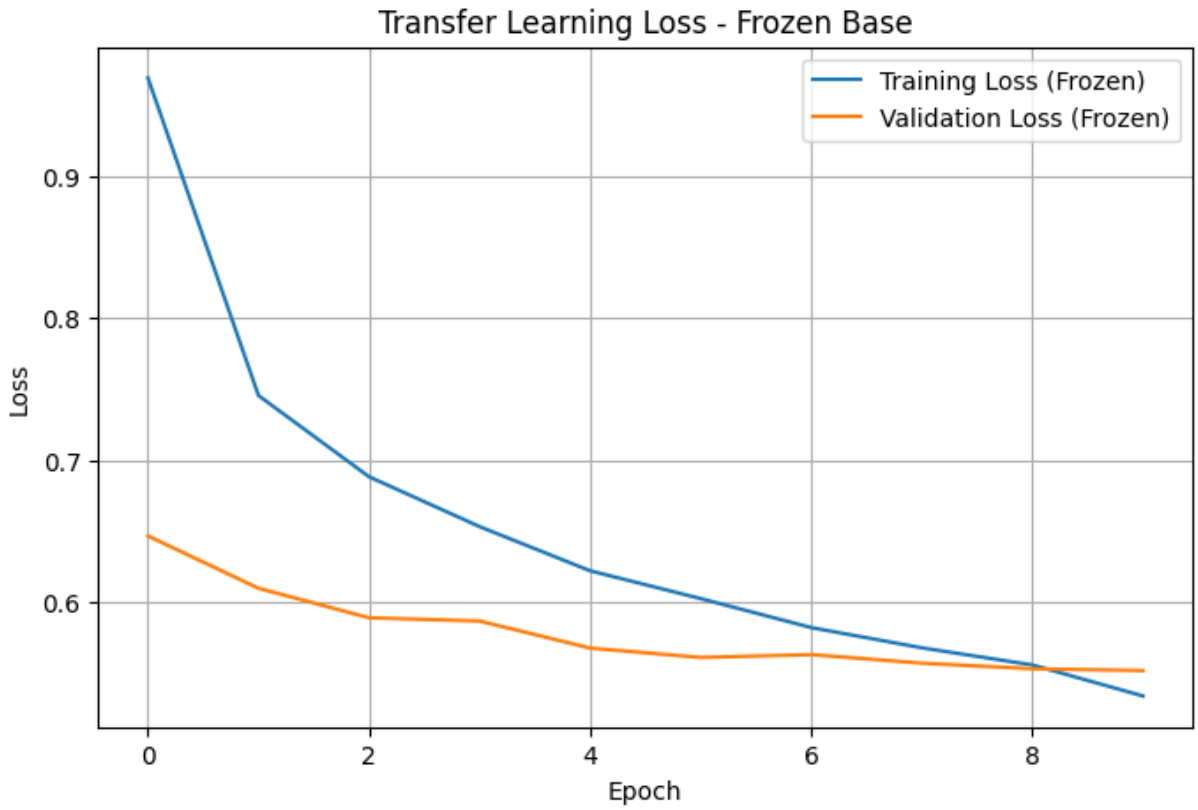
```
# 1. Evaluate on the resized test set
transfer_test_loss, transfer_test_accuracy = transfer_model.evaluat
    x_test_resized, y_test_cat, verbose=1
)

print("\nTransfer Learning Test Loss:", round(transfer_test_loss, 4
print("Transfer Learning Test Accuracy:", round(transfer_test_accur

# 2. Plot frozen-stage training history
plt.figure(figsize=(8, 5))
plt.plot(transfer_history.history['accuracy'], label='Training Accu
plt.plot(transfer_history.history['val_accuracy'], label='Validatio
plt.title('Transfer Learning Accuracy - Frozen Base')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(transfer_history.history['loss'], label='Training Loss (Fr
plt.plot(transfer_history.history['val_loss'], label='Validation Lo
plt.title('Transfer Learning Loss - Frozen Base')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()

# 3. Plot fine-tuning history
plt.figure(figsize=(8, 5))
plt.plot(fine_tune_history.history['accuracy'], label='Training Acc
plt.plot(fine_tune_history.history['val_accuracy'], label='Validati
plt.title('Transfer Learning Accuracy - Fine-Tuning')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(fine_tune_history.history['loss'], label='Training Loss (F
plt.plot(fine_tune_history.history['val_loss'], label='Validation L
plt.title('Transfer Learning Loss - Fine-Tuning')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
```

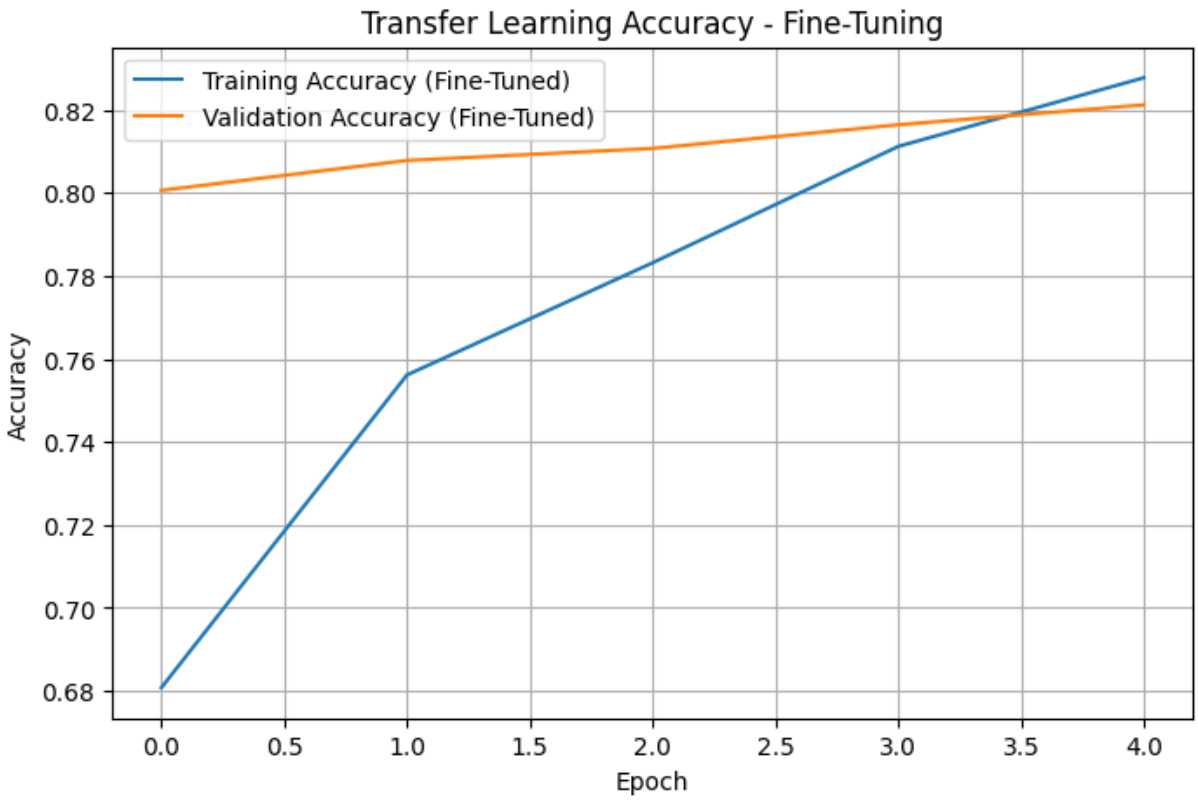
```
plt.show()

# 4. Predictions
transfer_pred_probs = transfer_model.predict(x_test_resized)
transfer_pred = np.argmax(transfer_pred_probs, axis=1)
y_true = y_test.flatten()

# 5. Classification report
print("\nClassification Report for Transfer Learning Model:\n")
print(classification_report(y_true, transfer_pred, target_names=cla

# 6. Confusion matrix
transfer_cm = confusion_matrix(y_true, transfer_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(
    transfer_cm,
    annot=True,
    fmt='d',
    cmap='Greens',
    xticklabels=class_names,
    yticklabels=class_names
)
plt.title('Confusion Matrix - Transfer Learning Model')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

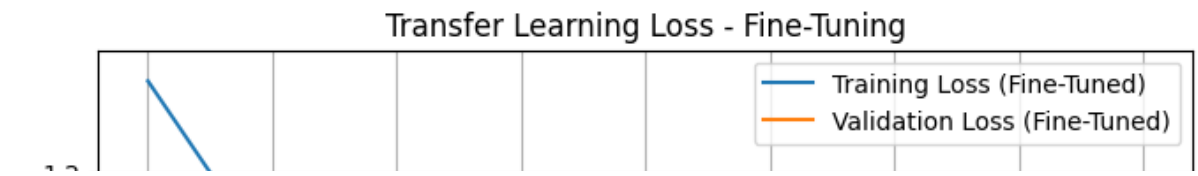
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 54s 173ms/step - accuracy: 0.8168 - los

Transfer Learning Test Loss: 0.5968
Transfer Learning Test Accuracy: 0.8168
```

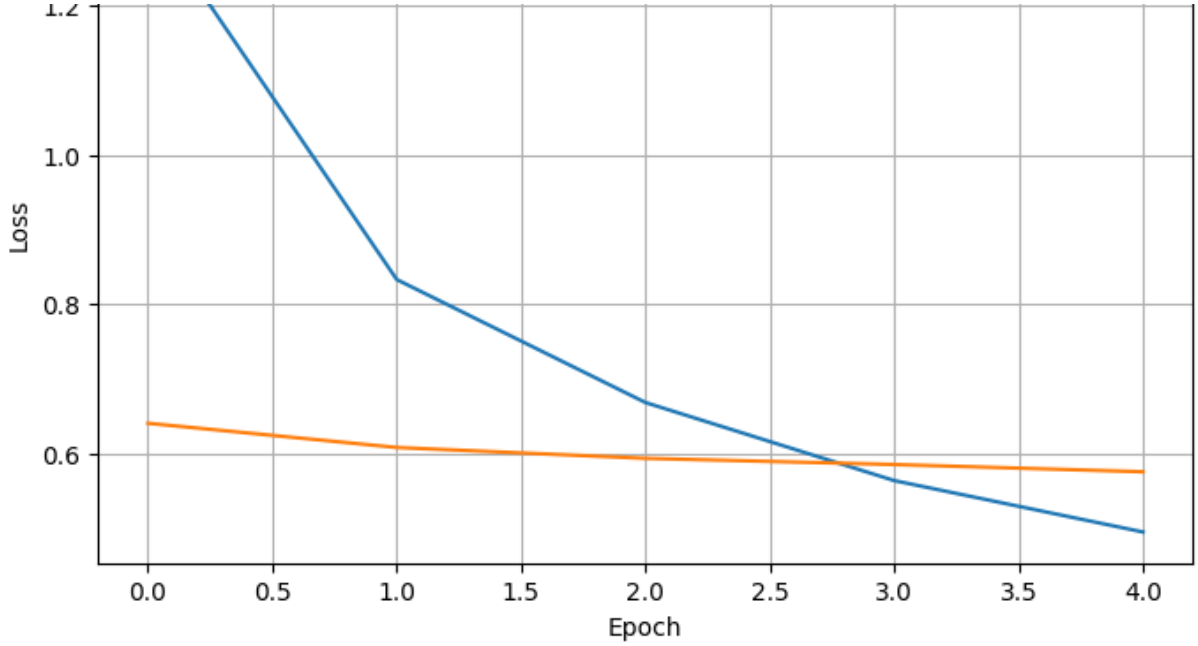


Validation Accuracy (Frozen)
0.66
0
2
4
6
8
Epoch

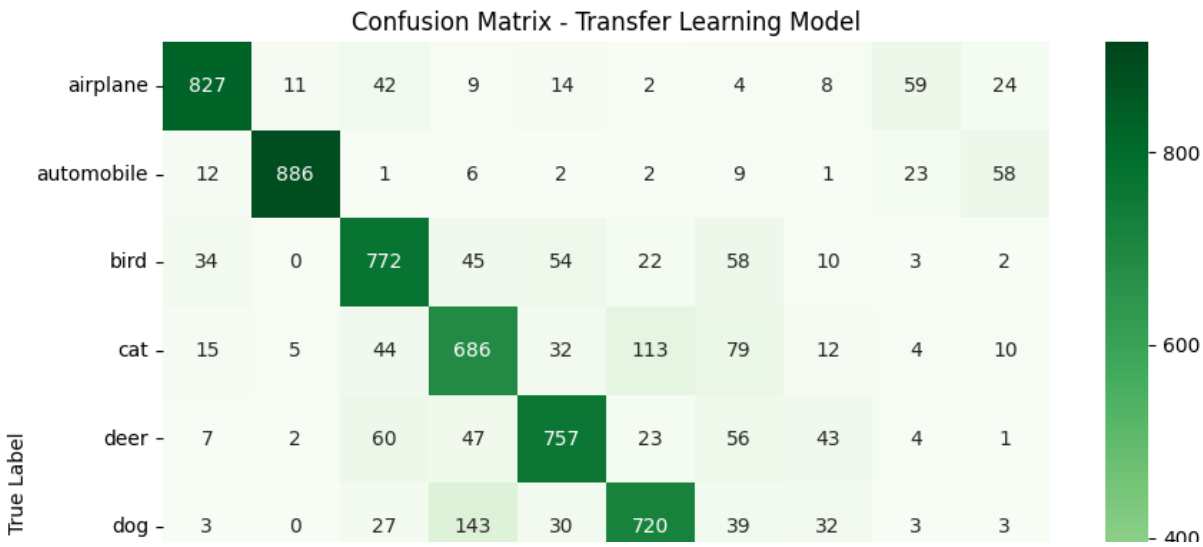

Transfer Learning Loss - Frozen Base
Training Loss (Frozen)
Validation Loss (Frozen)
Loss
0.9
0.8
0.7
0.6
0
2
4
6
8
Epoch

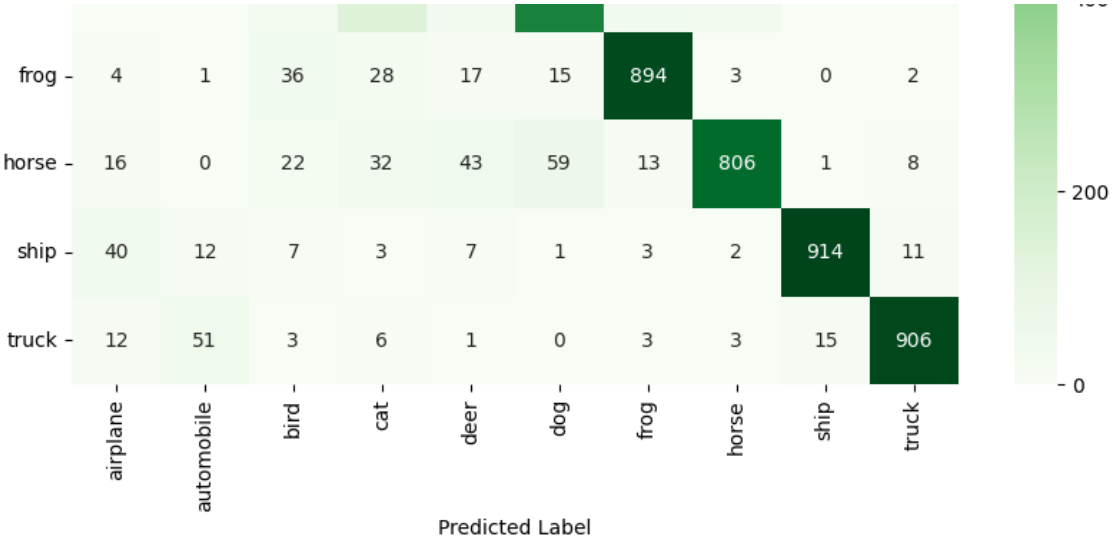

Transfer Learning Accuracy - Fine-Tuning
Training Accuracy (Fine-Tuned)
Validation Accuracy (Fine-Tuned)
Accuracy
0.82
0.80
0.78
0.76
0.74
0.72
0.70
0.68
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch


Transfer Learning Loss - Fine-Tuning
Training Loss (Fine-Tuned)
Validation Loss (Fine-Tuned)



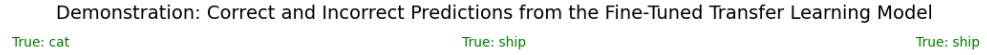

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 58s 180ms/step

Classification Report for Transfer Learning Model:

              precision    recall  f1-score   support

    airplane       0.85      0.83      0.84      1000
  automobile       0.92      0.89      0.90      1000
        bird       0.76      0.77      0.77      1000
         cat       0.68      0.69      0.68      1000
        deer       0.79      0.76      0.77      1000
         dog       0.75      0.72      0.74      1000
        frog       0.77      0.89      0.83      1000
       horse       0.88      0.81      0.84      1000
        ship       0.89      0.91      0.90      1000
       truck       0.88      0.91      0.89      1000

    accuracy                           0.82     10000
   macro avg       0.82      0.82      0.82     10000
weighted avg       0.82      0.82      0.82     10000
```

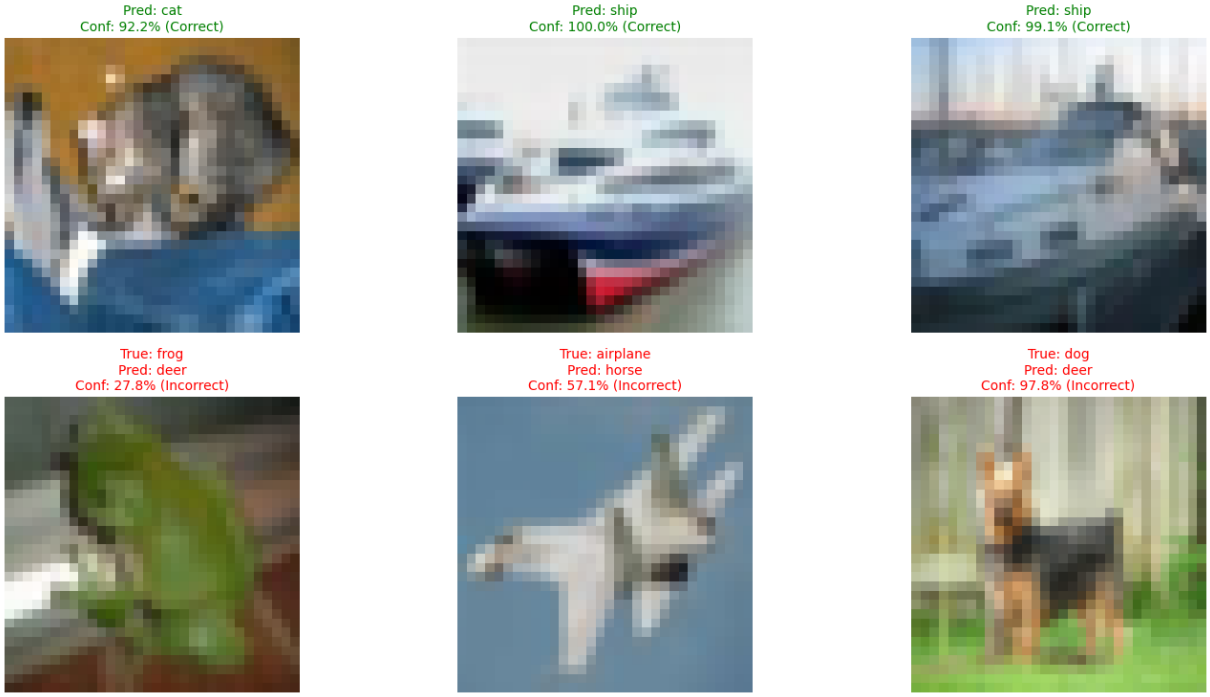





```
# ============================================================
# Section 7: Final Demonstration for Presentation
```

```
# Best Demo Version: 3 Correct + 3 Incorrect Predictions
# ============================================================

import matplotlib.pyplot as plt
import numpy as np

# We assume these already exist from earlier sections:
# transfer_pred_probs = transfer_model.predict(x_test_resized)
# transfer_pred = np.argmax(transfer_pred_probs, axis=1)
# y_true = y_test.flatten()

# 1. Find correct and incorrect prediction indices
correct_indices = np.where(transfer_pred == y_true)[0]
incorrect_indices = np.where(transfer_pred != y_true)[0]

# 2. Select examples
selected_correct = correct_indices[:3]
selected_incorrect = incorrect_indices[:3]

selected_indices = list(selected_correct) + list(selected_incorrect

# 3. Plot results
plt.figure(figsize=(15, 8))

for i, idx in enumerate(selected_indices):
    image = x_test[idx]
    true_label = class_names[y_true[idx]]
    pred_label = class_names[transfer_pred[idx]]
    confidence = np.max(transfer_pred_probs[idx]) * 100

    is_correct = pred_label == true_label
    title_color = "green" if is_correct else "red"
    result_text = "Correct" if is_correct else "Incorrect"

    plt.subplot(2, 3, i + 1)
    plt.imshow(image)
    plt.title(
        f"True: {true_label}\nPred: {pred_label}\nConf: {confidence
        color=title_color,
        fontsize=10
    )
    plt.axis("off")

plt.suptitle("Demonstration: Correct and Incorrect Predictions from
plt.tight_layout()
plt.show()
```




Pred: cat
Conf: 92.2% (Correct)
Pred: ship
Conf: 100.0% (Correct)
Pred: ship
Conf: 99.1% (Correct)
True: frog
Pred: deer
Conf: 27.8% (Incorrect)
True: airplane
Pred: horse
Conf: 57.1% (Incorrect)
True: dog
Pred: deer
Conf: 97.8% (Incorrect)

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.